

# Python Programming Assignment II

In Assignment I, you created an object-oriented program that analyzed simulated fitness or podcast-session measurements stored in Python lists and dictionaries. In this assignment, you will extend that design into a file-based application. Your program must load CSV data, validate records, continue safely when possible, organize code into modules and save useful reports.

Complete the same option you used in Assignment I.

## 3 Project options

| Option                                  | Continuation                                                                                    | Official files                                                       |
|-----------------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| A . Smart Fitness Session Analyzer      | Analyze wearable fitness sessions and recovery relative to participant reference values.        | participants.csv, fitness_sessions.csv, fitness_sessions_invalid.csv |
| B. Podcast Voice and Recording Analyzer | Analyze extracted acoustic features and recording quality relative to speaker reference values. | speakers.csv, recording_sessions.csv, recording_sessions_invalid.csv |

The supplied data are simulated. Do not rename columns or modify the official CSV files when demonstrating the final program. You may create additional files for your own tests.

## 4 Required functionality

### 4.1 Read profiles and sessions

- Load the profile file for the selected option.
- Load both the valid and intentionally invalid session files.
- Use with open(..., encoding="utf-8", newline="") and Python's csv module.
- Convert values to suitable types rather than leaving all measurements as strings.
- Group records by session or recording identifier.
- Connect each session to an existing participant or speaker.

### 4.2 Validate identifiers with regular expressions

Use full-match behaviour or anchored patterns. At minimum, validate the person identifier and the session or recording identifier.

| Value              | Required format            | Example pattern                 |
|--------------------|----------------------------|---------------------------------|
| Participant ID     | P followed by three digits | <code>^P\d{3}\$</code>          |
| Speaker ID         | S followed by three digits | <code>^S\d{3}\$</code>          |
| Fitness session ID | FIT-YYYY-NNN               | <code>^FIT-\d{4}-\d{3}\$</code> |

|              |              |                                 |
|--------------|--------------|---------------------------------|
| Recording ID | REC-YYYY-NNN | <code>^REC-\d{4}-\d{3}\$</code> |
|--------------|--------------|---------------------------------|

You may also validate filenames with a regular expression. Do not use regular expressions for ordinary numerical range checks.

### 4.3 Validate records

- Detect missing required fields and unexpected row lengths.
- Reject values that cannot be converted to the required type.
- Reject impossible or out-of-range measurements.
- Reject unknown participant or speaker identifiers.
- Treat poor signal quality as a data-quality issue according to a clearly documented rule.
- For the podcast option, handle windows where `speech_present` is `False`.
- Record the source filename, row number, field and reason for every rejected row.

### 4.4 Handle errors

Catch errors only where the program can add useful context or recover safely. The program should continue to the next row or file when appropriate. It must not hide programming errors behind a single broad exception.

- Create and use at least two custom exception classes.
- Handle `FileNotFoundError`, `PermissionError`, `ValueError`, `KeyError` and relevant csv errors where they may occur.
- Include useful information in error messages.
- Do not use an empty except block.
- Do not use except `Exception` as the main error-handling strategy.

```
class InvalidIdentifierError(ValueError):
    """Raised when an identifier has an invalid format."""

class InvalidRecordError(ValueError):
    """Raised when a CSV record cannot be accepted."""
```

### 4.5 Perform the analysis

Reuse and improve the core analysis from Assignment I. Calculate appropriate summaries, compare them with personal reference values, classify each session or recording, and provide reasons for the result. Your program must distinguish insufficient usable data from a meaningful normal or unusual result.

### 4.6 Create an output directory when it does not exist. Save at least the following files:

```
output/
├── analysis_summary.csv
├── analysis_report.txt
└── rejected_records.txt
```

- `analysis_summary.csv` contains one row per processed session or recording.
- `analysis_report.txt` gives a readable explanation of each result.
- `rejected_records.txt` identifies rejected rows and explains why they were rejected.
- Running the program again should produce predictable output without requiring manual cleanup.

## 5 Package and module design

Organize the application as a package.

Class and function names remain your design decision. A larger number of classes or modules does not automatically produce a better design.

## 6 Minimum technical requirements

- A Python package containing at least three meaningful modules.
- Continued use of classes, encapsulation and composition.
- At least two custom exception classes that are actually raised and handled.
- At least three targeted try and except structures.
- At least two meaningful regular-expression validations.
- Reading at least two official input files.
- Writing at least two output files.
- Use of pathlib or another reliable approach to file paths.
- A structured result represented by a dictionary or domain object.
- Tests covering valid, invalid, missing-file and boundary cases.
- No pandas, NumPy, scikit-learn, databases, external APIs, graphical interfaces or machine-learning models.

## 7 Program behaviour and command line

The final program must run from the repository root without changing source code or hard-coded absolute paths. Provide a simple command such as:

```
python3 main.py --profiles data/participants.csv --sessions data/fitness_sessions.csv --  
output output
```

An equivalent command for the podcast option is acceptable. If you implement command-line arguments, use only the Python standard library. The program should print a short completion summary showing accepted rows, rejected rows and created report files.